Gini Backend Skill Challenge
Payment Request Service

Thanks for your interest in the Senior Backend Developer role at Gini! This challenge is a small, realistic slice of the kind of work our backend team does every day. It is not a puzzle and there is no hidden trick — we want to see how you design, build, and reason about a small service under real-world constraints.

The rules of the game (please read first)

- **Time budget: the core task is scoped at roughly 2–3 hours.** You choose your own total budget. When you hit it, stop — and instead write down in the README what you would do next and what you deliberately cut. We grade decisions, not volume. A smaller solution with clear reasoning beats a big one without it.
- **AI tools are allowed** — **and we ask you to be open about it.** Using Claude, ChatGPT, Copilot etc. is normal engineering practice in 2026 and we won't penalize it. What we care about is *your* judgment. Your README must include a short **AI usage note** (see below), and in the follow-up interview you will walk us through your code, change it live, and defend your decisions. Submit only code you fully understand and would put your name on in a production code review.
- **Tech constraints**: Java 21 (or 17+), Spring Boot 3.x, Gradle preferred (Maven is fine). These mirror our actual stack. Everything else — libraries, database, structure — is your call, and your calls are part of what we evaluate.
- **Delivery** — via a private Git repository, either way works:
  - **(a)** Send us your GitHub handle and we prepare a private repository for you to push to, or
  - **(b)** create a **private repository** on your own GitHub account and grant read access to the GitHub handle(s) we sent you along with this challenge.

Please keep the code private — also after the process — since we reuse this exercise. And push your real commit history, not one squashed commit: we like reading it — it shows how you work.

Gini GmbH
Ridlerstr. 57
80339 München

E-Mail: hello@gini.net
Telefon: +49 89 215 285 00
Web: https://www.gini.net

Geschäftsführer:
Alexander Jäger
Registergericht: HRB 193307
Amtsgericht München
USt.-ID.: DE 27931366

Bankverbindung:
Deutsche Bank PGK München
IBAN: DE96 7007 0024 0013 7778 00
BIC: DEUTDEDBMUC

Background

At Gini, our services let banking apps turn documents into payments. For this challenge you build a simplified, standalone Payment Request Service: it manages requests for payment that a business sends to a customer ("please pay €49.90 for invoice 2026-0815").

Core task (required, ~2–3 hours)

Build a REST service that manages payment requests.

Functional requirements

1. **Create a payment request.** A payment request has at least: recipient name, recipient IBAN, amount + currency, purpose text. It starts in status OPEN.
2. **Fetch a payment request** by its ID, including its current status.
3. **Settle a payment request** (mark it paid). Only an OPEN request can be settled, and it must be settled **exactly once**. (In real life settlement comes from a payment event — for the core task a simple endpoint is fine.)

The parts that actually matter

These are deliberately the hard 20% — they are what we will talk about in the interview:

- **Validation**. Reject garbage with useful error responses: invalid IBAN, non-positive or absurd amounts, missing fields. Using a library (or a simple checksum implementation) for the IBAN is perfectly fine — how you shape the error responses is the interesting part.
- **Idempotent creation**. Clients (mobile apps on flaky networks) retry requests. The same create request sent twice must not produce two payment requests. Design how the client and server achieve this and document it.
- **Settle exactly once, even under concurrency.** Two settle calls for the same request may arrive at the same time (a retry, a duplicate event). Exactly one must win; the other caller must get a meaningful response — and the request must never be settled twice. Explain in the README why your approach is safe with more than one service instance running.
- **Persistence**. Use a relational database via a real repository layer (JPA, Spring Data JDBC, jOOQ — your choice). An embedded/in-memory DB (H2, or Postgres via Testcontainers) is perfectly fine; we care about the data model and access patterns, not ops setup.
- **Tests**. Meaningful ones — **3 to 5 focused tests are plenty.** At minimum, the settle rules and the idempotency behavior should be covered. We'd rather see 5 tests that test behavior than 50 that test mocks.

README (required, ~half a page is enough)

- How to run the service and the tests.
- Your key design decisions and trade-offs (e.g., how idempotency works, how the concurrency case is handled, API shape choices).
- What you cut or would do next with more time.

Gini GmbH
Ridlerstr. 57
80339 München

E-Mail: hello@gini.net
Telefon: +49 89 215 285 00
Web: https://www.gini.net

Geschäftsführer:
Alexander Jäger
Registergericht: HRB 193307
Amtsgericht München
USt.-ID.: DE 27931366

Bankverbindung:
Deutsche Bank PGK München
IBAN: DE96 7007 0024 0013 7778 00
BIC: DEUTDEDBMUC

- **AI usage note**: which AI tools you used and for what, at least one thing they got wrong or you overrode, and how you verified the generated parts. ("I didn't use any" is also a fine answer.)

Optional extensions (pick at most 2 — or none)

Only if you have budget left, pick what interests you most. **Which ones you pick (and skip) is itself a signal we'll ask about.** Don't do more than two.

- **A. Messaging**: consume a payment.settled event from RabbitMQ instead of (or in addition to) the settle endpoint. Show how you handle duplicate delivery and a "poison" message that keeps failing.
- **B. Security**: protect the API with token-based auth (OAuth2 client credentials, or a documented simplification). Explain what you'd do differently in production.
- **C. Concurrency proof**: demonstrate the double-settle race is handled with a test that actually exercises it.
- **D. Operability**: health/readiness endpoints suitable for Kubernetes, sensible metrics, structured logs — and a note on which of these you'd alert on.
- **E. CI**: a GitHub Actions workflow that builds and tests the project.
- **F. List endpoint done properly**: pagination, filtering by status, stable ordering.
- **G. Cancellation**: add a cancel operation (only OPEN requests can be cancelled) — including making the cancel-vs-settle race safe.

What happens next

If we move forward, there are two remote or on-site **technical interview** sessions:

1. **Hands-on session (45 min)** — starts from your submission: you'll walk us through it, we'll ask "why" a lot, and we may extend or change it together live. Then a code review exercise (you review a small pull request the way you would at work).
2. **Technical interview (up to 90 min)** — a conversation about engineering fundamentals, your experience and the systems you'd own. And plenty of time for your questions to us.

Questions about the challenge? Ask us — asking good questions is not a penalty, it's a habit we like.

If you have any questions regarding the coding challenge, please contact **Jue** at **jue@gini.net**.

For any other questions throughout the hiring process, feel free to reach out to **Jules** at **julia.partusch@gini.net**.

Good luck, and have fun!

Gini GmbH
Ridlerstr. 57
80339 München

E-Mail: hello@gini.net
Telefon: +49 89 215 285 00
Web: https://www.gini.net

Geschäftsführer:
Alexander Jäger
Registergericht: HRB 193307
Amtsgericht München
USt.-ID.: DE 27931366

Bankverbindung:
Deutsche Bank PGK München
IBAN: DE96 7007 0024 0013 7778 00
BIC: DEUTDEDBMUC